

Lattice algorithms for sparse secret LWE

Elena Kirshanova

based on joint works with

Alexander Karenin

Alexander May

Julian Nowakowski

Eamonn W. Postlethwaite

Ludo N. Pulles,

Fernando Virdia,

Paul Vié

at ACCESS seminar

June 2, 2026



The talk is based on two papers

1. **Fast Slicer for Batch-CVP: Making Lattice Hybrid Attacks Practical.**
Together with Alexander Karenin, Alexander May, Julian Nowakowski.
AsiaCrypt 2025
2. **Cool + Cruel = Dual and New Benchmarks for Sparse LWE.**
Together with Alexander Karenin, Julian Nowakowski, Eamonn W. Postlethwaite,
Ludo N. Pulles, Fernando Virdia, Paul Vié.
EuroCrypt 2026

What is sparse LWE?

In LWE we are given:

$$\mathbf{A} \in \mathbb{Z}_q^{m \times n} \quad \text{and} \quad \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \bmod q$$

To find:

$$\mathbf{s} \in \mathbb{Z}_q$$

In **sparse** LWE $\mathbf{s}$ has low Hamming weight and small entries.

Examples:

- ▶ ML-KEM secret come from central binomial distribution
- ▶ Efficient FHE schemes use binary or ternary secrets of low Hamming weight (e.g., CKKS uses $n = 2^{16}$ and ternary secrets of Hamming weight 32.)
- ▶ NTRU KEM uses ternary secrets with relative Hamming weight $\sim 2/3$.

LWE is average-case Bounded Distance Decoding (BDD)

In BDD we are given:

- ▶ basis $\mathbf{B}$ of a lattice $\mathcal{L}(\mathbf{B}) := \mathbf{B} \cdot \mathbb{Z}^d$
- ▶ $\mathbf{t} = \mathbf{B}\mathbf{x} + \mathbf{y}$ for some $\mathbf{x} \in \mathbb{Z}^d$ and small $\|\mathbf{y}\|$

To find:

- ▶ the unique $\mathbf{x} \in \mathbb{Z}^d$ minimizing $\|\mathbf{t} - \mathbf{B}\mathbf{x}\|$

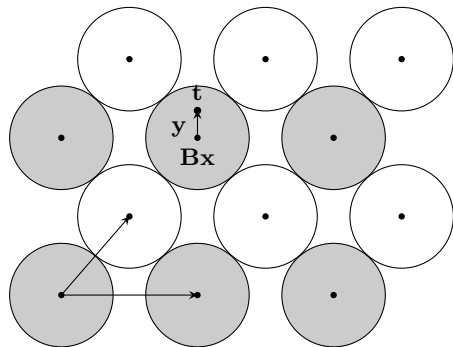
LWE is average-case Bounded Distance Decoding (BDD)

In BDD we are given:

- ▶ basis $\mathbf{B}$ of a lattice $\mathcal{L}(\mathbf{B}) := \mathbf{B} \cdot \mathbb{Z}^d$
- ▶ $\mathbf{t} = \mathbf{B}\mathbf{x} + \mathbf{y}$ for some $\mathbf{x} \in \mathbb{Z}^d$ and small $\|\mathbf{y}\|$

To find:

- ▶ the unique $\mathbf{x} \in \mathbb{Z}^d$ minimizing $\|\mathbf{t} - \mathbf{B}\mathbf{x}\|$



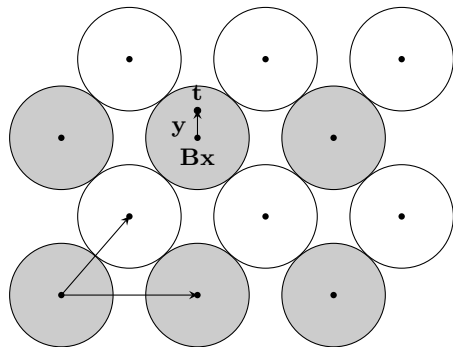
LWE is average-case Bounded Distance Decoding (BDD)

In BDD we are given:

- ▶ basis $\mathbf{B}$ of a lattice $\mathcal{L}(\mathbf{B}) := \mathbf{B} \cdot \mathbb{Z}^d$
- ▶ $\mathbf{t} = \mathbf{B}\mathbf{x} + \mathbf{y}$ for some $\mathbf{x} \in \mathbb{Z}^d$ and small $\|\mathbf{y}\|$

To find:

- ▶ the unique $\mathbf{x} \in \mathbb{Z}^d$ minimizing $\|\mathbf{t} - \mathbf{B}\mathbf{x}\|$



For LWE:

$$\mathcal{L}(\mathbf{B}) = \{(\mathbf{v}_1, \mathbf{v}_2) \in \mathbb{Z}^n \times \mathbb{Z}^m \mid \mathbf{v}_2 = -\mathbf{A}\mathbf{v}_1\}$$

$$\mathbf{B}\mathbf{x} = (-\mathbf{s}, \mathbf{b} - \mathbf{e}) \quad \mathbf{y} = (\mathbf{s}, \mathbf{e}) \quad \mathbf{t} = (0^n, \mathbf{b})$$

For sparse secret LWE, only few candidates for $\mathbf{B}\mathbf{x}$!

$\implies$ hybrid attacks = enumerate $\mathbf{s}$ partially + lattice reduction

Hybrid attacks on LWE

$$\mathbf{t} = \mathbf{B}\mathbf{x} + \mathbf{y} - \text{our BDD instance.}$$

Step 1:

1. Split the instance $\mathbf{t} = \mathbf{B}_0\mathbf{x}_0 + \mathbf{B}_1\mathbf{x}_1 + \mathbf{y}$ for $\mathbf{B} = [\mathbf{B}_0|\mathbf{B}_1]$
2. Enumerate a list of $\mathbf{t}^{(i)} := \mathbf{t} - \mathbf{B}_1\mathbf{x}_1^{(i)}$

Hybrid attacks on LWE

$$\mathbf{t} = \mathbf{B}\mathbf{x} + \mathbf{y} - \text{our BDD instance.}$$

Step 1:

1. Split the instance $\mathbf{t} = \mathbf{B}_0\mathbf{x}_0 + \mathbf{B}_1\mathbf{x}_1 + \mathbf{y}$ for $\mathbf{B} = [\mathbf{B}_0|\mathbf{B}_1]$
2. Enumerate a list of $\mathbf{t}^{(i)} := \mathbf{t} - \mathbf{B}_1\mathbf{x}_1^{(i)}$

Step 2:

1. Find i^* such that $\mathbf{t}^{(i^*)} = \mathbf{B}_0\mathbf{x}_0 + \mathbf{y}$
2. This gives a lower-dimensional BDD instance on $\mathcal{L}(\mathbf{B}_0)$

Hybrid attacks on LWE

$\mathbf{t} = \mathbf{B}\mathbf{x} + \mathbf{y}$ – our BDD instance.

Step 1:

1. Split the instance $\mathbf{t} = \mathbf{B}_0\mathbf{x}_0 + \mathbf{B}_1\mathbf{x}_1 + \mathbf{y}$ for $\mathbf{B} = [\mathbf{B}_0|\mathbf{B}_1]$
2. Enumerate a list of $\mathbf{t}^{(i)} := \mathbf{t} - \mathbf{B}_1\mathbf{x}_1^{(i)}$

This is where Primal and Dual attacks differ

Step 2:

1. Find i^* such that $\mathbf{t}^{(i^*)} = \mathbf{B}_0\mathbf{x}_0 + \mathbf{y}$

2. This gives a lower-dimensional BDD instance on $\mathcal{L}(\mathbf{B}_0)$

Hybrid attacks on LWE

$\mathbf{t} = \mathbf{B}\mathbf{x} + \mathbf{y}$ – our BDD instance.

Step 1:

1. Split the instance $\mathbf{t} = \mathbf{B}_0\mathbf{x}_0 + \mathbf{B}_1\mathbf{x}_1 + \mathbf{y}$ for $\mathbf{B} = [\mathbf{B}_0|\mathbf{B}_1]$
2. Enumerate a list of $\mathbf{t}^{(i)} := \mathbf{t} - \mathbf{B}_1\mathbf{x}_1^{(i)}$

This is where Primal and Dual attacks differ

Step 2:

1. Find i^* such that $\mathbf{t}^{(i^*)} = \mathbf{B}_0\mathbf{x}_0 + \mathbf{y}$

2. This gives a lower-dimensional BDD instance on $\mathcal{L}(\mathbf{B}_0)$

Primal

- ▶ Run a BDD solver for every $\mathbf{t}^{(i)}$
- ▶ A solution gives candidates $\mathbf{x}_0^{(i)}$ for $\mathbf{x}_0$
- ▶ Take i that minimizes

$$\|\mathbf{t} - \mathbf{B}(\mathbf{x}_0^{(i)}, \mathbf{x}_1^{(i)})\|$$

Hybrid attacks on LWE

$\mathbf{t} = \mathbf{B}\mathbf{x} + \mathbf{y}$ – our BDD instance.

Step 1:

1. Split the instance $\mathbf{t} = \mathbf{B}_0\mathbf{x}_0 + \mathbf{B}_1\mathbf{x}_1 + \mathbf{y}$ for $\mathbf{B} = [\mathbf{B}_0|\mathbf{B}_1]$
2. Enumerate a list of $\mathbf{t}^{(i)} := \mathbf{t} - \mathbf{B}_1\mathbf{x}_1^{(i)}$

This is where Primal and Dual attacks differ

Step 2:

1. Find i^* such that $\mathbf{t}^{(i^*)} = \mathbf{B}_0\mathbf{x}_0 + \mathbf{y}$
2. This gives a lower-dimensional BDD instance on $\mathcal{L}(\mathbf{B}_0)$

Primal

- ▶ Run a BDD solver for every $\mathbf{t}^{(i)}$
- ▶ A solution gives candidates $\mathbf{x}_0^{(i)}$ for $\mathbf{x}_0$
- ▶ Take i that minimizes

$$\|\mathbf{t} - \mathbf{B}(\mathbf{x}_0^{(i)}, \mathbf{x}_1^{(i)})\|$$

Dual

- ▶ Compute many short vectors $\mathbf{w} \in \mathcal{L}(\mathbf{B}_0)^*$
- ▶ Apply a stat. test to $\langle \mathbf{t}^{(i)}, \mathbf{w} \rangle$
- ▶ Idea:
 - ▶ $\langle \mathbf{t}^{(i)}, \mathbf{w} \rangle$ tells $\text{dist}(\mathbf{t}^{(i)}, \mathcal{L}(\mathbf{B}_0))$
 - ▶ $\text{dist}(\mathbf{t}^{(i^*)}, \mathcal{L}(\mathbf{B}_0)) < \text{dist}(\mathbf{t}^{(i)}, \mathcal{L}(\mathbf{B}_0))$ for $i \neq i^*$.

Hybrid attacks on LWE

$\mathbf{t} = \mathbf{B}\mathbf{x} + \mathbf{y}$ – our BDD instance.

Step 1:

1. Split the instance $\mathbf{t} = \mathbf{B}_0\mathbf{x}_0 + \mathbf{B}_1\mathbf{x}_1 + \mathbf{y}$ for $\mathbf{B} = [\mathbf{B}_0|\mathbf{B}_1]$
2. Enumerate a list of $\mathbf{t}^{(i)} := \mathbf{t} - \mathbf{B}_1\mathbf{x}_1^{(i)}$

This is where Primal and Dual attacks differ

Step 2:

1. Find i^* such that $\mathbf{t}^{(i^*)} = \mathbf{B}_0\mathbf{x}_0 + \mathbf{y}$

2. This gives a lower-dimensional BDD instance on $\mathcal{L}(\mathbf{B}_0)$

Primal

- ▶ Run a BDD solver for every $\mathbf{t}^{(i)}$
- ▶ A solution gives candidates $\mathbf{x}_0^{(i)}$ for $\mathbf{x}_0$
- ▶ Take i that minimizes

$$\|\mathbf{t} - \mathbf{B}(\mathbf{x}_0^{(i)}, \mathbf{x}_1^{(i)})\|$$

Dual

- ▶ Compute many short vectors $\mathbf{w} \in \mathcal{L}(\mathbf{B}_0)^*$
- ▶ Apply a stat. test to $\langle \mathbf{t}^{(i)}, \mathbf{w} \rangle$
- ▶ Idea:
 - ▶ $\langle \mathbf{t}^{(i)}, \mathbf{w} \rangle$ tells $\text{dist}(\mathbf{t}^{(i)}, \mathcal{L}(\mathbf{B}_0))$
 - ▶ $\text{dist}(\mathbf{t}^{(i^*)}, \mathcal{L}(\mathbf{B}_0)) < \text{dist}(\mathbf{t}^{(i)}, \mathcal{L}(\mathbf{B}_0))$ for $i \neq i^*$.

Tail-BDD

- ▶ $\pi_{d-\beta}^\perp()$ – projection orthogonal to $\text{Span}(\mathbf{B}[: d - \beta])$ for some $\beta < d$.
- ▶ $\mathcal{L}(\mathbf{B}_{[d-\beta, d)}) := \pi_{d-\beta}^\perp(\mathbf{B})$.

¹Here $\mathbf{B}$ should be understood as $\mathbf{B}_0$ from the prev. slide

Tail-BDD

- ▶ $\pi_{d-\beta}^\perp()$ – projection orthogonal to $\text{Span}(\mathbf{B}[: d - \beta])$ for some $\beta < d$.
- ▶ $\mathcal{L}(\mathbf{B}_{[d-\beta, d)}) := \pi_{d-\beta}^\perp(\mathbf{B})$.

Idea: BDD over d -dim. $\mathcal{L}(\mathbf{B}) = \text{CVP over } \beta\text{-dim. } \mathcal{L}(\mathbf{B}_{[d-\beta, d)})$.¹

$$\mathbf{t} = \mathbf{B}\mathbf{x} + \mathbf{y} = \mathbf{B}(\mathbf{x}_0, \mathbf{x}_1) + \mathbf{y} \text{ for } \mathbf{x} = (\mathbf{x}_0, \mathbf{x}_1) \in \mathbb{Z}^{d-\beta} \times \mathbb{Z}^\beta.$$

Tail-BDD:

1. Run lattice reduction on $\mathcal{L}(\mathbf{B})$ with block-size β
2. Run CVP solver on $\mathcal{L}(\mathbf{B}_{[d-\beta, d)})$ with target $\pi_{d-\beta}^\perp(\mathbf{t})$:

$$\pi_{d-\beta}^\perp(\mathbf{t}) = \mathcal{L}(\mathbf{B}_{[d-\beta, d)})\mathbf{x}_1 + \pi_{d-\beta}^\perp(\mathbf{y})$$

¹Here $\mathbf{B}$ should be understood as $\mathbf{B}_0$ from the prev. slide

Tail-BDD

- ▶ $\pi_{d-\beta}^\perp()$ – projection orthogonal to $\text{Span}(\mathbf{B}[: d - \beta])$ for some $\beta < d$.
- ▶ $\mathcal{L}(\mathbf{B}_{[d-\beta, d]}) := \pi_{d-\beta}^\perp(\mathbf{B})$.

Idea: BDD over d -dim. $\mathcal{L}(\mathbf{B}) = \text{CVP over } \beta\text{-dim. } \mathcal{L}(\mathbf{B}_{[d-\beta, d]})$.¹

$$\mathbf{t} = \mathbf{B}\mathbf{x} + \mathbf{y} = \mathbf{B}(\mathbf{x}_0, \mathbf{x}_1) + \mathbf{y} \text{ for } \mathbf{x} = (\mathbf{x}_0, \mathbf{x}_1) \in \mathbb{Z}^{d-\beta} \times \mathbb{Z}^\beta.$$

Tail-BDD:

1. Run lattice reduction on $\mathcal{L}(\mathbf{B})$ with block-size β
2. Run CVP solver on $\mathcal{L}(\mathbf{B}_{[d-\beta, d]})$ with target $\pi_{d-\beta}^\perp(\mathbf{t})$:

$$\begin{aligned} \pi_{d-\beta}^\perp(\mathbf{t}) &= \mathcal{L}(\mathbf{B}_{[d-\beta, d]})\mathbf{x}_1 + \pi_{d-\beta}^\perp(\mathbf{y}) \\ &\uparrow \\ \text{Close to } \mathcal{L}(\mathbf{B}_{[d-\beta, d]}) &\text{ if } \|\pi_{d-\beta}^\perp(\mathbf{y})\| < \frac{1}{2}\|\mathbf{b}_{d-\beta}^\star\| \end{aligned}$$

¹Here $\mathbf{B}$ should be understood as $\mathbf{B}_0$ from the prev. slide

How large β should be?

Estimate block-size β such that $\|\pi_{d-\beta}^\perp(\mathbf{y})\| < \frac{1}{2}\|\mathbf{b}_{d-\beta}^\star\|$.²

[KKMN25]

For a BDD instance $(\mathcal{L}(\mathbf{B}), \mathbf{t})$, TailBDD outputs the correct solution in time $2^{0.292\beta}$ if $\mathbf{B}$ is BKZ β -reduced with

$$\beta = \frac{d}{1 + 2 \log_d \left(\frac{\lambda_1(\mathcal{L}(\mathbf{B}))}{2\|\mathbf{y}\|} \right)}.$$

This is essentially the same result as we have for solving BDD via Kannan's embedding.

²For average-case analysis replace $1/2$ with 1 .

How large β should be?

Estimate block-size β such that $\|\pi_{d-\beta}^\perp(\mathbf{y})\| < \frac{1}{2}\|\mathbf{b}_{d-\beta}^\star\|$.²

[KKMN25]

For a BDD instance $(\mathcal{L}(\mathbf{B}), \mathbf{t})$, TailBDD outputs the correct solution in time $2^{0.292\beta}$ if $\mathbf{B}$ is BKZ β -reduced with

$$\beta = \frac{d}{1 + 2 \log_d \left(\frac{\lambda_1(\mathcal{L}(\mathbf{B}))}{2\|\mathbf{y}\|} \right)}.$$

This is essentially the same result as we have for solving BDD via Kannan's embedding.

But, we have many BDD targets $\mathbf{t}$ for the same $\mathcal{L}(\mathbf{B})$.

²For average-case analysis replace $1/2$ with 1 .

Batch Tail-BDD

Observation: we have many BDD/CVP targets $\mathbf{t}^{(i)} := \mathbf{t} - \mathbf{B}_1 \mathbf{x}_1^{(i)}$.

The Randomized Slicer can solve several CVP instances efficiently **in batches**.

Randomized Slicer for Batch-CVP, [DLdW19, DLvW20]

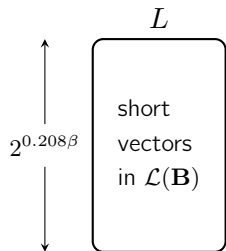
The Randomized Slicer heuristically solves M CVP instances on the same lattice of dimension β , in time and space

$$T_{\text{Slicer}} = \max\{2^{0.292\beta}, M \cdot 2^{0.234\beta}\}.$$

If $M \leq 2^{0.058\beta}$, solving M instances of CVP is essentially equivalent to running SVP on β -dim. lattice.

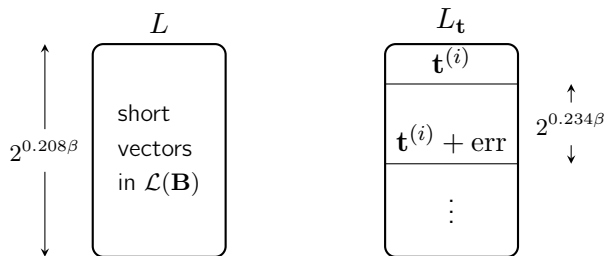
A Glimpse at Randomized Slicer [DLdW19, DLvW20]

$$\mathbf{t}^{(i)} = \mathbf{B}\mathbf{x}^{(i)} + \mathbf{y}^{(i)} - \text{CVP instances on } \mathcal{L}(\mathbf{B}) \subset \mathbb{R}^\beta$$



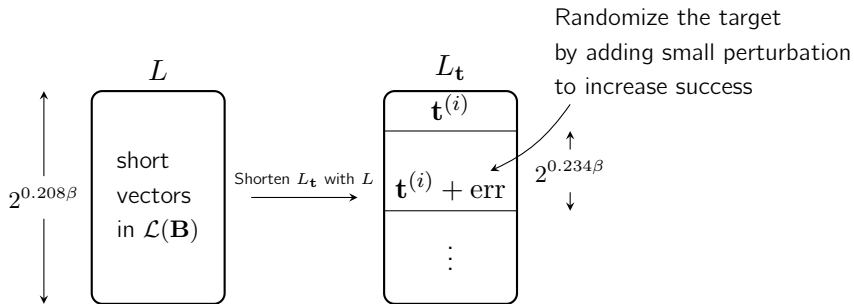
A Glimpse at Randomized Slicer [DLdW19, DLvW20]

$$\mathbf{t}^{(i)} = \mathbf{B}\mathbf{x}^{(i)} + \mathbf{y}^{(i)} - \text{CVP instances on } \mathcal{L}(\mathbf{B}) \subset \mathbb{R}^\beta$$



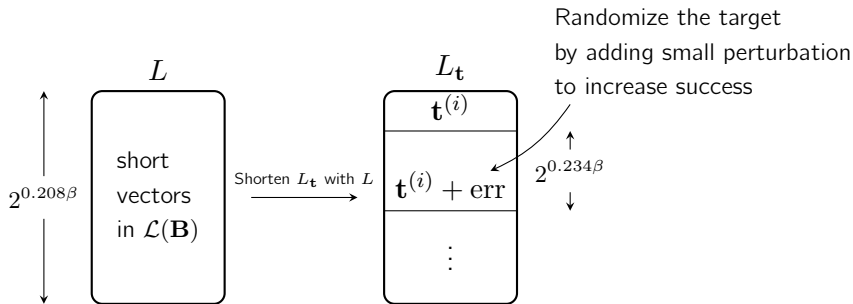
A Glimpse at Randomized Slicer [DLdW19, DLvW20]

$$\mathbf{t}^{(i)} = \mathbf{B}\mathbf{x}^{(i)} + \mathbf{y}^{(i)} - \text{CVP instances on } \mathcal{L}(\mathbf{B}) \subset \mathbb{R}^\beta$$



A Glimpse at Randomized Slicer [DLdW19, DLvW20]

$$\mathbf{t}^{(i)} = \mathbf{B}\mathbf{x}^{(i)} + \mathbf{y}^{(i)} - \text{CVP instances on } \mathcal{L}(\mathbf{B}) \subset \mathbb{R}^\beta$$



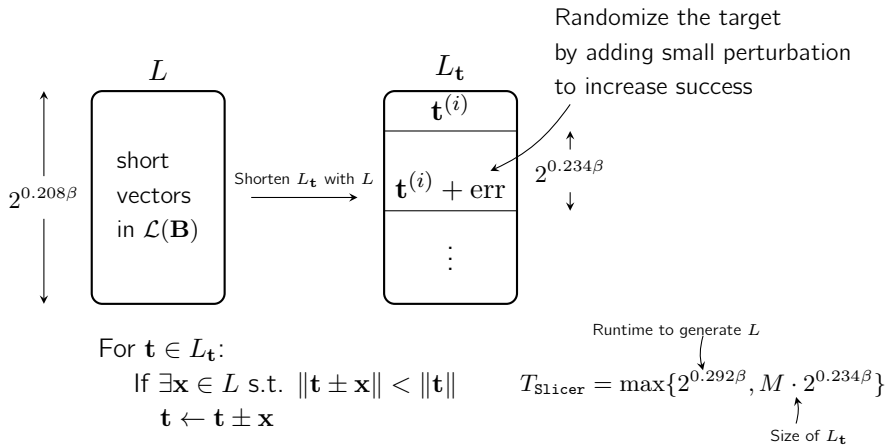
For $\mathbf{t} \in L_{\mathbf{t}}$:

If $\exists \mathbf{x} \in L$ s.t. $\|\mathbf{t} \pm \mathbf{x}\| < \|\mathbf{t}\|$

$\mathbf{t} \leftarrow \mathbf{t} \pm \mathbf{x}$

A Glimpse at Randomized Slicer [DLdW19, DLvW20]

$$\mathbf{t}^{(i)} = \mathbf{B}\mathbf{x}^{(i)} + \mathbf{y}^{(i)} - \text{CVP instances on } \mathcal{L}(\mathbf{B}) \subset \mathbb{R}^\beta$$



Total cost of Primal Hybrid

Step 1:

1. Split the instance $\mathbf{t} = \mathbf{B}_0\mathbf{x}_0 + \mathbf{B}_1\mathbf{x}_1 + \mathbf{y}$

2. Enumerate a list of $\mathbf{t}^{(i)} := \mathbf{t} - \mathbf{B}_1\mathbf{x}_1^{(i)}$

Step 2:

1. Find i^* such that $\mathbf{t}^{(i^*)} = \mathbf{B}_0\mathbf{x}_0 + \mathbf{y}$

2. This gives a lower-dimensional BDD instance on $\mathcal{L}(\mathbf{B}_0)$

Complexity of Step 1:

We enumerate $\mathbf{y}_1 \leftarrow D^\kappa$,³

D is a distribution of BDD error.

Enumerating

$$M = 2^{\mathcal{H}(D)\kappa+1}$$

targets gives constant

success probability [GMN26].

Here $\mathcal{H}()$ is Shannon entropy.

³We really enumerate $\pi_{d-\kappa}^\perp(\mathbf{y}) = (0^{d-\kappa}, y_{d-\kappa}, \dots, y_{d-1})$ for Batch Tail-BDD and split $\mathbf{B}$ into $\mathbf{B}_0$ and $\mathbf{B}_{[d-\kappa, d]}$

Total cost of Primal Hybrid

Step 1:

1. Split the instance $\mathbf{t} = \mathbf{B}_0\mathbf{x}_0 + \mathbf{B}_1\mathbf{x}_1 + \mathbf{y}$
2. Enumerate a list of $\mathbf{t}^{(i)} := \mathbf{t} - \mathbf{B}_1\mathbf{x}_1^{(i)}$

Step 2:

1. Find i^* such that $\mathbf{t}^{(i^*)} = \mathbf{B}_0\mathbf{x}_0 + \mathbf{y}$
2. This gives a lower-dimensional BDD instance on $\mathcal{L}(\mathbf{B}_0)$

Complexity of Step 2:

We BKZ reduce $\mathcal{L}(\mathbf{B}_0)$ with

$$\beta = \frac{d-\kappa}{1+2\log_{d-\kappa}\left(\frac{\lambda_1(\mathcal{L}(\mathbf{B}_0))}{2\|\mathbf{y}\|}\right)}$$

Run the Randomized slicer on a sublattice of $\mathcal{L}(\mathbf{B}_0)$ and all $\mathbf{t}^{(i)}$

Total cost of Primal Hybrid

Step 1:

1. Split the instance $\mathbf{t} = \mathbf{B}_0\mathbf{x}_0 + \mathbf{B}_1\mathbf{x}_1 + \mathbf{y}$
2. Enumerate a list of $\mathbf{t}^{(i)} := \mathbf{t} - \mathbf{B}_1\mathbf{x}_1^{(i)}$

Step 2:

1. Find i^* such that $\mathbf{t}^{(i^*)} = \mathbf{B}_0\mathbf{x}_0 + \mathbf{y}$
2. This gives a lower-dimensional BDD instance on $\mathcal{L}(\mathbf{B}_0)$

Complexity of Step 2.1:

Lift a CVP solution from a sublattice of $\mathcal{L}(\mathbf{B}_0)$ to $\mathcal{L}(\mathbf{B}_0)$ using Babai's lift.

This is $\text{poly}(d)$ time.

Total cost of Primal Hybrid

Step 1:

1. Split the instance $\mathbf{t} = \mathbf{B}_0\mathbf{x}_0 + \mathbf{B}_1\mathbf{x}_1 + \mathbf{y}$
2. Enumerate a list of $\mathbf{t}^{(i)} := \mathbf{t} - \mathbf{B}_1\mathbf{x}_1^{(i)}$

Step 2:

1. Find i^* such that $\mathbf{t}^{(i^*)} = \mathbf{B}_0\mathbf{x}_0 + \mathbf{y}$
2. This gives a lower-dimensional BDD instance on $\mathcal{L}(\mathbf{B}_0)$

To balance the cost of BKZ- β with Randomized Slicer on $M = 2^{\mathcal{H}(D)\kappa+1}$ targets, set

$$\kappa = \frac{\mathcal{B}}{\mathcal{B} + \frac{\mathcal{H}(D)}{0.058}} \cdot d, \quad \text{where } \mathcal{B} = \frac{1}{1 + 2 \log_d \left(\frac{\lambda_1(\mathcal{L}(\mathbf{B}))}{2\|\mathbf{y}\|} \right)}$$

Main result: asymptotic of hybrid primal

Primal Hybrid solves a d -dim. BDD instance $\mathbf{B}, \mathbf{t} = \mathbf{B}\mathbf{x} + \mathbf{y}$, where $\mathbf{y} \leftarrow D^d$, in time

$$2^{0.292(\mathcal{B}-\mathcal{K})d},$$

where

$$\mathcal{B} = \frac{1}{1 + 2 \log_d \left(\frac{\lambda_1(\mathcal{L}(\mathbf{B}))}{2\|\mathbf{y}\|} \right)}, \quad \mathcal{K} = \frac{\mathcal{B}^2}{\mathcal{B} + \frac{\mathcal{H}(D)}{0.058}}, \quad \mathcal{H} - \text{Shannon entropy.}$$

Main result: asymptotic of hybrid primal

Primal Hybrid solves a d -dim. BDD instance $\mathbf{B}, \mathbf{t} = \mathbf{B}\mathbf{x} + \mathbf{y}$, where $\mathbf{y} \leftarrow D^d$, in time

$$2^{0.292(\mathcal{B}-\mathcal{K})d},$$

where

$$\mathcal{B} = \frac{1}{1 + 2 \log_d \left(\frac{\lambda_1(\mathcal{L}(\mathbf{B}))}{2\|\mathbf{y}\|} \right)}, \quad \mathcal{K} = \frac{\mathcal{B}^2}{\mathcal{B} + \frac{\mathcal{H}(D)}{0.058}}, \quad \mathcal{H} - \text{Shannon entropy.}$$

For $\text{Binom}(p)$ – binomial with parameter p and $\mathcal{T}(p)$ – ternary of expected relative weight p :

D	Binom(1)	Binom(2)	Binom(3)	$\mathcal{T}(\frac{2}{3})$	$\mathcal{T}(\frac{1}{3})$	$\mathcal{T}(\frac{1}{6})$
$\mathcal{H}(D)$	1.500	2.030	2.333	1.584	1.251	0.816
$0.292\mathcal{K}$	0.010	0.008	0.007	0.010	0.012	0.019

Primal Hybrid in Practice

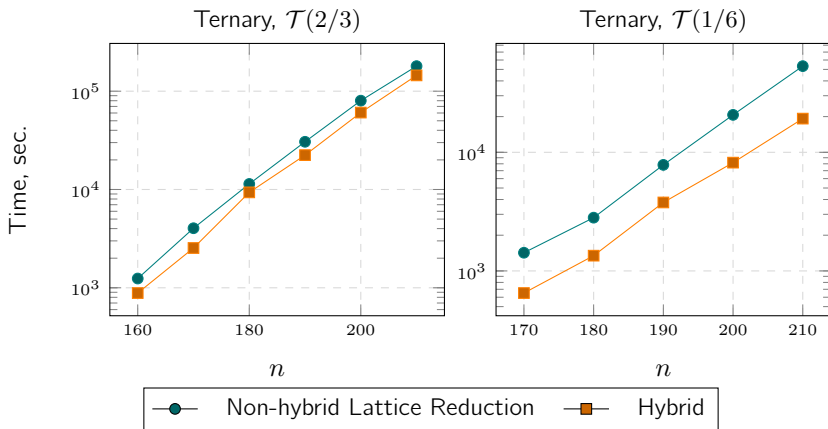


Figure: Timings of the Pure Lattice attack vs. timings of the Hybrid attack on LWE instances of dimension n .

Next Steps

- ▶ Randomized Slicer for codes? We have sieving for codes ([GJN24, DEEK24]), but no analogue for a BDD solver
- ▶ Realistic (practical and reasonably grounded) meet-in-the-middle speed-up for enumeration?
- ▶ Engineering effort: there is a lot of room for implementation speed-ups.

Really Sparse Secrets

- ▶ Considered for benchmarking in Wenger, Saxena, Malhou, Thieu, Lauter **Benchmarking Attacks on Learning with Errors**. S&P'25.
- ▶ Example LWE parameters: $n = 512$, $q \approx 2^{28}$, Hamming weight of **s** in $\{20, 21, 25\}$.
- ▶ [WSMTL, S&P'25] benchmarks suggest ($A > B$ means B outperforms A):

Primal Attack > SALSA (ML-based attacks) > Cool+Cruel

[Kannan'83]

[WCCL'22, LSWMGCL'23, LWACL'23]

[NMWSLCL'24]

Really Sparse Secrets

- ▶ Considered for benchmarking in Wenger, Saxena, Malhou, Thieu, Lauter **Benchmarking Attacks on Learning with Errors**. S&P'25.
- ▶ Example LWE parameters: $n = 512, q \approx 2^{28}$, Hamming weight of **s** in $\{20, 21, 25\}$.
- ▶ [WSMTL, S&P'25] benchmarks suggest ($A > B$ means B outperforms A):

Primal Attack $>$ SALSA (ML-based attacks) $>$ Cool+Cruel
[Kannan'83] [WCCL'22, LSWMGCL'23, LWACL'23] [NMWSLCL'24]

Guess+Verify $<$ Dual $\overline{\text{attack}}$
[ACW'19] [Albrecht'17]

Our work [KKNPPVV26]

Special case of hybrid: Guess+Verify

- ▶ For extremely sparse LWE/BDD, we can afford large guessing dimension κ at low cost
- ▶ This leaves us with a much smaller dimensional lattice $\mathbf{B}_0$
- ▶ Instead of running expensive Randomized Slicer, Guess+Verify runs Babai Lift to verify the guess.
- ▶ A clever implementation of Babai Lift allows for some batching
- ▶ Guess+Verify has been already improved in
<https://eprint.iacr.org/2026/279>,
<https://eprint.iacr.org/2026/366.pdf>.

Next Steps

- ▶ Are there LWE/BDD regimes where dual outperforms (asymptotically and in practice) primal?
- ▶ Efficient implementation of dual attack?

References

- ▶ [\[DEEK24\]](#) Ducas L., Esser A., Etinski S., Kirshanova E. Asymptotics and Improvements of Sieving for Codes. 2024
- ▶ [\[DLdW19\]](#) Doulgerakis E., Laarhoven T., de Weger B. Finding closest lattice vectors using approximate Voronoi cells. 2019
- ▶ [\[DLvW20\]](#) Ducas L., Laarhoven T., van Woerden W.P.J. The randomized slicer for CVPP: Sharper, faster, smaller, batchier. 2020
- ▶ [\[GJN24\]](#) Guo, Q., Johansson, T., Nguyen, V. A new sieving-style information-set decoding algorithm. 2024
- ▶ [\[GMN24\]](#) Glaser T., May A., Nowakowski J.. Entropy suffices for guessing most keys. 2026
- ▶ [\[NMWSLCL24\]](#) N. Nolte, M. Malhou, E. Wenger, S. Stevens, C. Yuanchen Li, F. Charton, and K. E. Lauter. The cool and the cruel: Separating hard parts of LWE secrets.
- ▶ [\[WSMTL\]](#) E. Wenger, E. Saxena, M. Malhou, E. Thieu, and K. Lauter. Benchmarking Attacks on Learning with Errors.